Recap: RNN (循环神经网络)

从"一对一"的输入/输出转向序列

{一对多: 图像字幕
多对一: 视频分类.

Today: Attention + Transformers

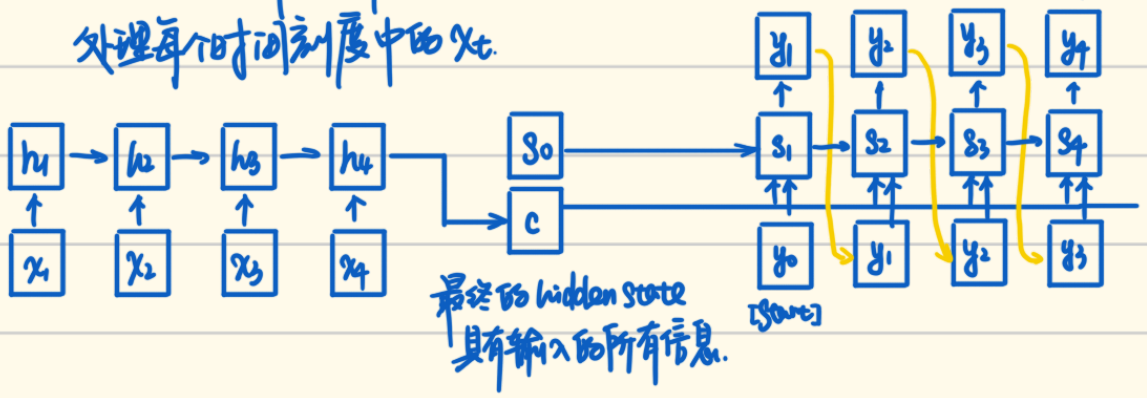
※ 在今天运行的所有最先进的神经网络都以transformer作为架构.

Attention引入: RNN处理序列-序列问题: Encoder - Decoder.

{输入: Sequence $x_1, \dots, x_T$ (例如机器翻译模型)
输出: Sequence $y_1, \dots, y_T$.

① Encoder: $h_t = f_W(h_{t-1}, x_t)$.

处理每个时间刻度中的 x_t.



最终的 hidden state 具有输入的所有信息.

② Decoder: $s_t = g_U(y_{t-1}, s_{t-1}, c)$

输入序列瓶颈.

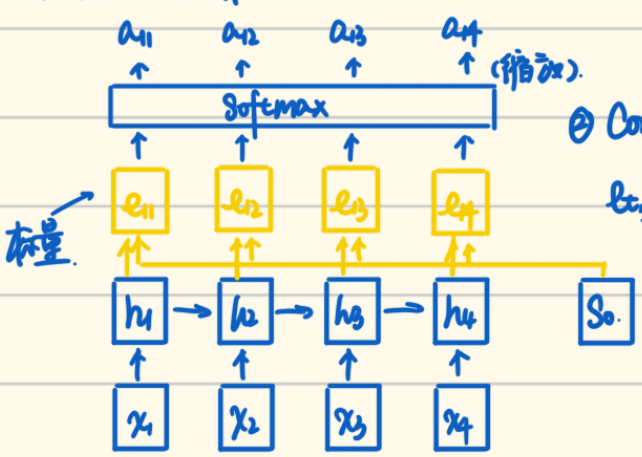
Problem: Input sequence bottlenecks.

"c": 上下文变量, 其大小恒定

Solution: 不通过固定大小的 context 向量来总结整个 input. 而是给予输出模型"回看"输入序列的能力. ⇓

Attention.

RNN with attention

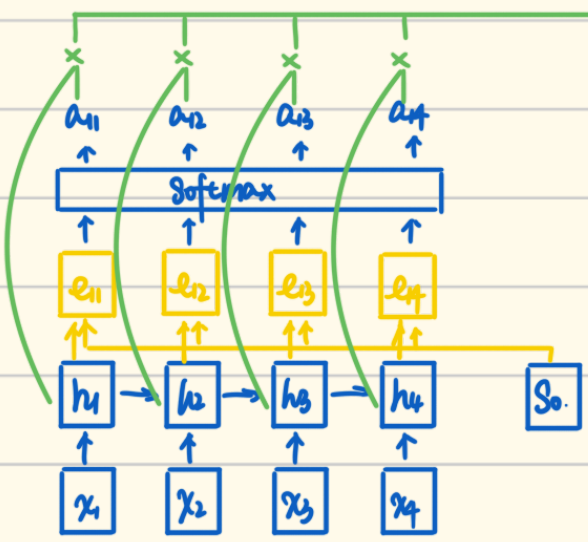


② Compute alignment scores.

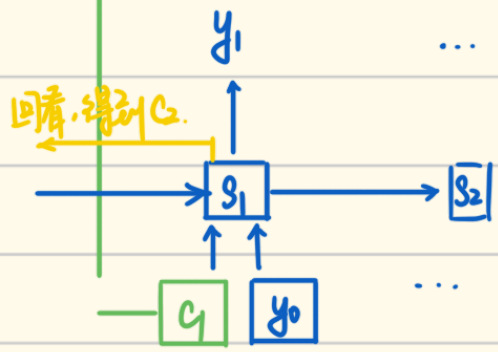
$e_{t,i} = f_{att}(s_{t-1}, h_i)$ f_{att}: 线性层.

① Encoder (RNN).

加权计算隐藏状态.



③ Decode.



"大型计算图":

整个结构都可自我训练.

设置decoder的初始状态:

{① encoder的最终状态.
② 全零

...

总结：attention机制：（in RNN）.

① 在每个Decode的时间步中，都尝试去回顾输入序列，加和得到上下文变量.

② 与之前架构相比：上下文+hiddenState+上一步输出 → 下一步输出

③ 解决"瓶颈"问题.

④ 通过attention路径也给了研究者了解模型运作的窗口.

普遍性的，强大的Attention.

Query vector
查询变量
（类似RNN中的Decoder的隐藏状态）.

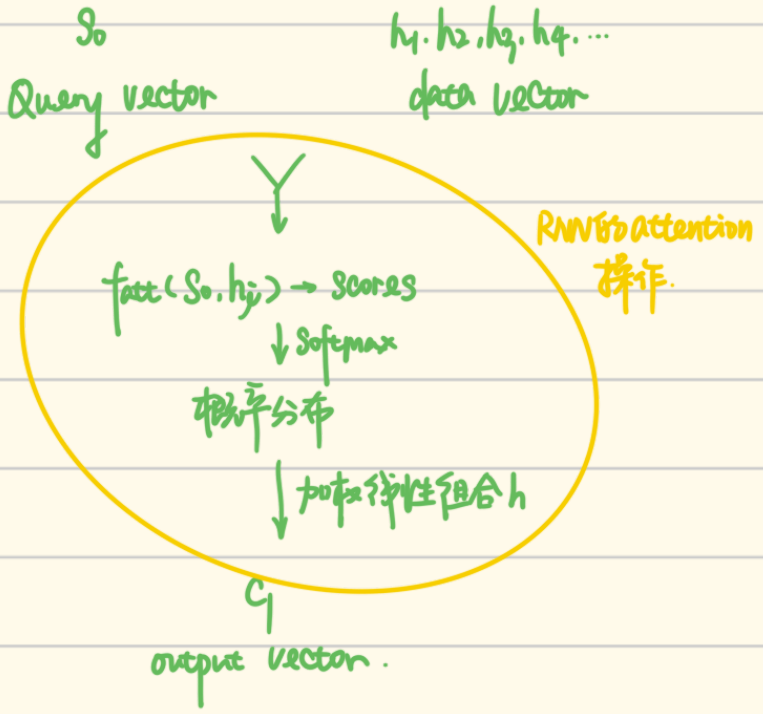
→

data vector
（要查询的数据）.
（类似RNN中Encoder的隐藏状态）.

output →

output vector.
（Context states）.

整个attention过程是：以某种新的方式回到数据，总结并生成上下文.



$(Q: [D_Q]$
$X: [N_X \times D_Q]$

计算过程：

相似度：点积来替代f_{att}？

① Similarities: $e_i = f_{att}(q \cdot x_i)$

② Attention weights: $a = Softmax(e)$

③ Output vector: $y = \sum a_i x_i$

→ ① $e_i = q \cdot x_i / \sqrt{D_Q}$ *避免梯度消失

使用点积简化f_{att}

② 扩展Q：$[N_Q \times D_Q]$.

$X: [N_X \times D_Q]$

→ $E = QX^T / \sqrt{D_Q}$ $[N_Q \times N_X]$

$E_{ij} = Q_i \cdot X_j / \sqrt{D_Q}$.

X^T在计算图中有复合功能：既用于计算"相似度"，又用于data本身.

↳ $A = Softmax(E, dim=1)$

↳ $Y = AX$ $[N_Q \times D_X]$ $(Y_i = \sum_j A_{ij} X_j)$

③ $X\ [N_X \times D_Q]$

W_K → K　　W_V → V

{ key matrix $W_K\ [D_X \times D_Q]$
value matrix $W_V\ [D_X \times D_Q]$.

添加 key 和 value 作为可学习参数，
将 N_X 个输入投影到两个功能上.

<直觉来自传统搜索引擎>

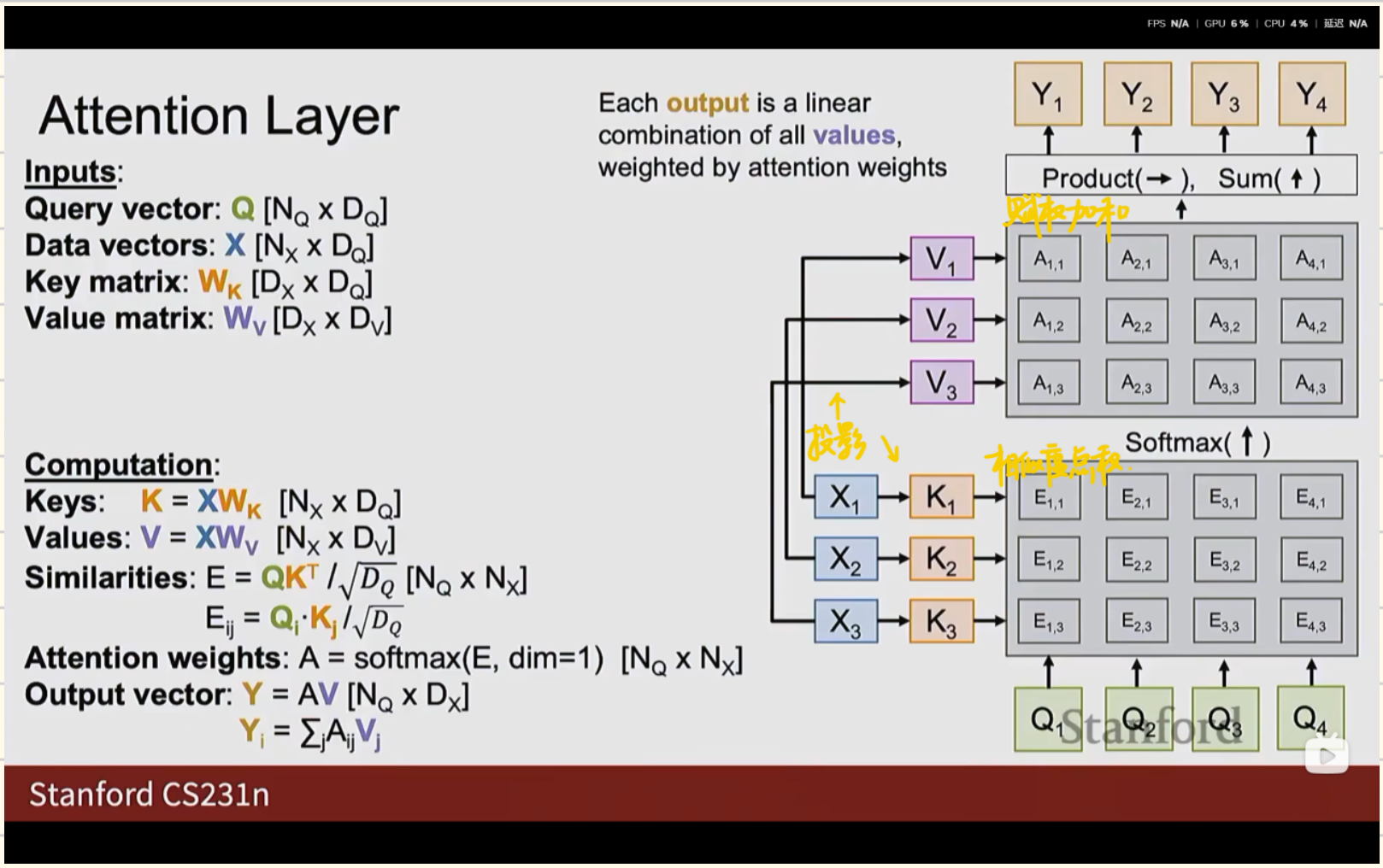
计算：
$$\begin{cases} K = XW_K & [N_X \times D_Q] \\ V = XW_V & [N_X \times D_V] \end{cases}$$

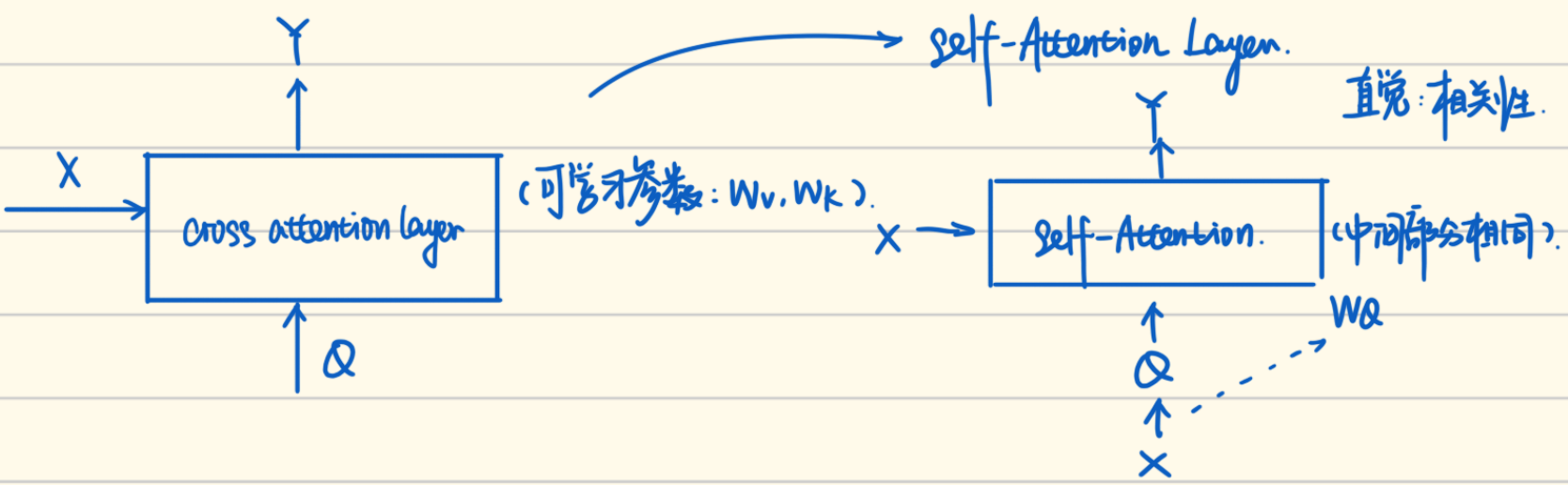
⇓

① Similarities: $E = QK^T/\sqrt{D_Q}$ $[N_Q \times N_X]$ $(E_{ij} = Q_i \cdot K_j/\sqrt{D_Q})$.

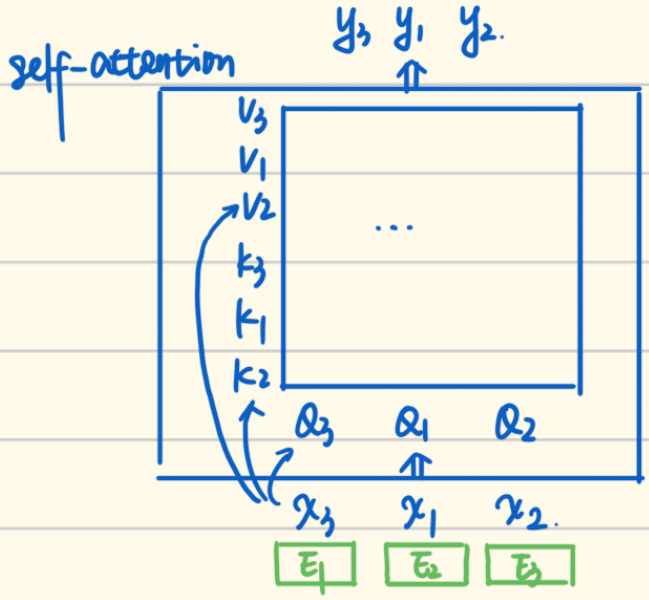
② Attention weights: $A = \text{Softmax}(E, \text{dim}=1)$

③ Output vector: $Y = AV$ $[N_Q \times D_V]$. $(Y_i = \sum_j A_{ij} V_j)$





特点：置换同变（self-attention）

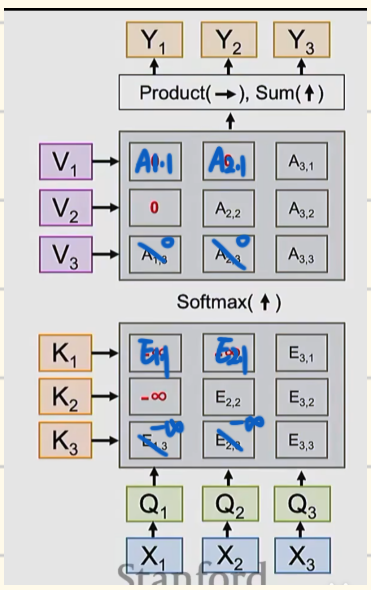


置换输入的顺序，
也仅会置换输出的顺序，
不会对结果有影响

对self-attention的直觉：
无序向量集，输出不取决于顺序

位置嵌入：强制给输入 x 增加一个 index 维度。

Masked Self-Attention Layer：

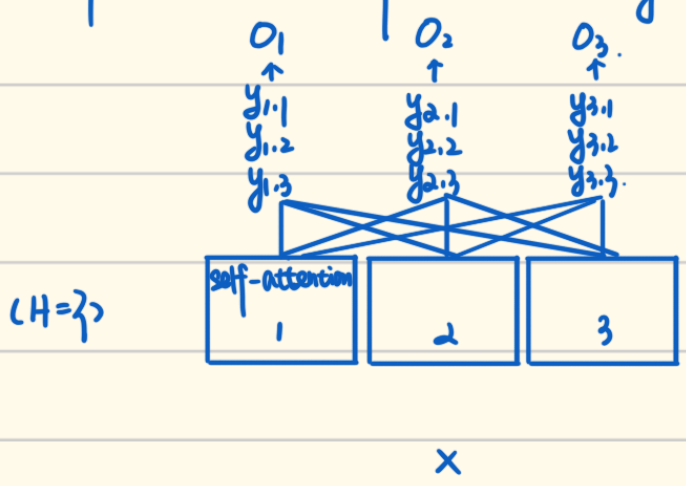


强制让 $E_{i,j}$ $(i<j)$ 变为负无穷，使注意力失效。

只能看见前面的，不能看见后面的。

（用于：语言翻译，序列 …）。

Multiheaded Self-Attention Layer 多头自注意力。



数据更多（在实践中的表明）。

X

Three Ways of Processing Sequences.

① RNN (1-D)
- 强大
- 不可并行，非并行。

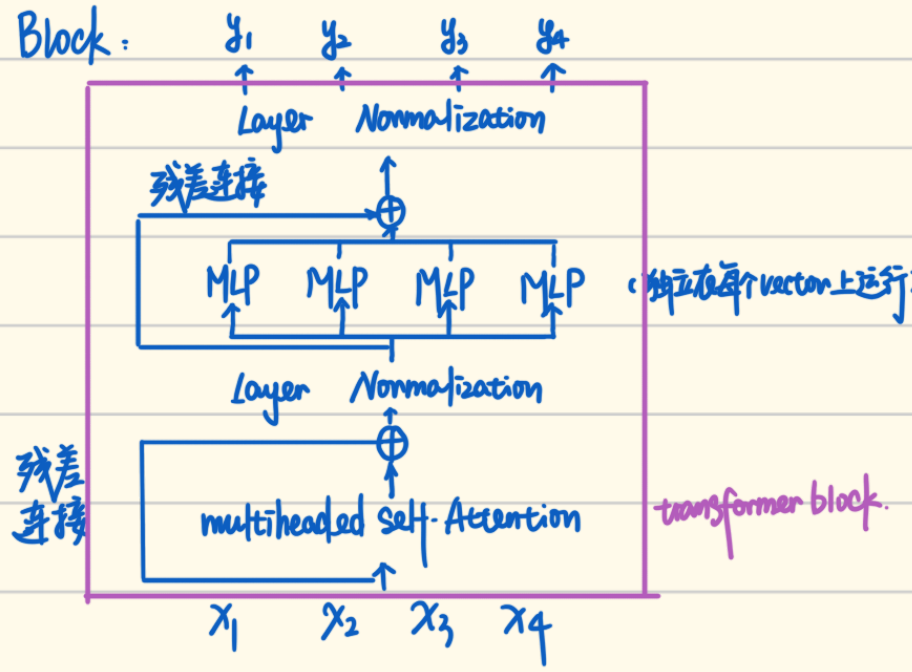
② Convolution (N-D)
- 对长序列无效，建立感受野很慢。
- 可并行。

③ Self-Attention
- 可用于长序列，可高度并行（4个矩阵乘法）
- 昂贵：$O(n^2)$

Attention is All You Need.

The Transformer：将self-attention置于核心的神经网络架构。

Transformer Block：



transformer block.

The transformer：

